

Analisi delle Prestazioni di Inserimento in Alberi Binari di Ricerca

Londero Stefano
172585

Grasso Federico
173731

Algoritmi e Strutture Dati - 2026

1 Introduzione

L'obiettivo del progetto è confrontare tra Alberi Binari di Ricerca (BST), Alberi (AVL) e Red-Black (RB) il tempo medio di inserimento di una chiave quando l'albero contiene n chiavi e, ottenere per diversi valori di n , stime robuste dei tempi di esecuzione.

2 Richiami Teorici

Il costo dell'inserimento in un albero di ricerca dipende dalla lunghezza del cammino radice-foglia, ed è quindi proporzionale alla sua altezza h .

- **BST Semplice:** Non garantisce alcun bilanciamento. Nel caso peggiore può degenerare in $\mathcal{O}(n)$, portando a tempi di inserimento lineari.
- **Alberi AVL:** Mantengono un bilanciamento stretto garantendo che la differenza di altezza tra i sotto-alberi di ogni nodo sia al massimo 1. Nel caso peggiore vengono assicurati tempi asintotici $\mathcal{O}(\log n)$ al costo di frequenti rotazioni strutturali.
- **Alberi Red-Black:** Utilizzando colorazioni per i nodi viene garantita una complessità $\Theta(\log n)$. Sebbene il cammino massimo possa essere leggermente più lungo rispetto a un AVL, il RBT richiede tipicamente meno operazioni di rotazione per ripristinare le proprietà dopo un inserimento.

3 Struttura del Progetto

Il progetto è stato strutturato nel seguente modo, seguendo un'ottica di programmazione ad oggetti:

- **Codici delle Strutture Dati:** I file `BST.py`, `AVL.py` e `RBT.py` contengono i codici completi delle strutture dati. In particolare nei codici delle ultime due strutture dati il codice è stato alleggerito riciclando alcune parti del codice da `BST.py`.
- **KeyManager:** Questo file implementa la gestione controllata delle chiavi tramite il partizionamento di un array, garantendo la selezione di chiavi esterne all'albero e occupate (interne) in un tempo $\mathcal{O}(1)$.
- **Project:** Il file `Project.py` segue il Design Pattern *Factory*, utilizzato nella programmazione ad oggetti. Questo file consente di configurare comodamente il tipo di albero tramite stringhe identificative ("BST", "AVL", "RBT").
- **Experiment Setup & Runner:** `ExperimentSetup.py` genera la sequenza di dimensioni N basata su una progressione geometrica di valori di n da testare (da 1000 a 10.000.000 elementi) e si occupa del riempimento iniziale dell'albero. Mentre `ExperimentRunner.py` implementa l'effettivo ciclo di misurazione e calcolando la mediana finale.

4 Algoritmo

Per ciascun valore di n identificato dalla progressione geometrica, e per ogni struttura dati analizzata, il processo di misurazione si articola rigorosamente nei seguenti passi:

1. **Inizializzazione:** Viene istanziato un albero inizialmente vuoto (BST, AVL o RBT).
2. **Popolazione iniziale:** Sfruttando `KeyManager`, l'albero viene popolato inserendo n chiavi estratte casualmente dall'insieme globale. Al termine di questa fase, l'albero raggiunge l'esatta dimensione n , lasciando nel compartimento dell'array una sola chiave disponibile e non ancora utilizzata.
3. **Ciclo di misurazione:** Viene avviato un blocco ripetuto di 10 iterazioni (configurabile). Ciascuna iterazione esegue in sequenza le seguenti operazioni:
 - (a) **Selezione ed Inserimento Cronometrato:** Viene prelevata l'unica chiave disponibile rimasta esclusa dall'albero e si misura in modo isolato il solo tempo di esecuzione dell'operazione `insert(k)` tramite la funzione di sistema `time.perf_counter()`. In questo istante la dimensione dell'albero sale temporaneamente a $n + 1$.
 - (b) **Rimozione ed Espulsione Silenziosa:** Si sceglie in modo puramente casuale una delle chiavi attualmente presenti all'interno dell'albero e la si rimuove tramite l'operazione `remove()` *senza cronometrare* l'esecuzione.
4. **Calcolo della statistica robusta:** Al termine delle 10 iterazioni, l'insieme dei tempi campionati viene aggregato calcolandone la mediana. Questo valore finale viene registrato come stima robusta delle prestazioni di inserimento per la dimensione n corrente.

Questa specifica tecnica a coppie di *insert-remove* garantisce il soddisfacimento simultaneo di due requisiti sperimentali critici:

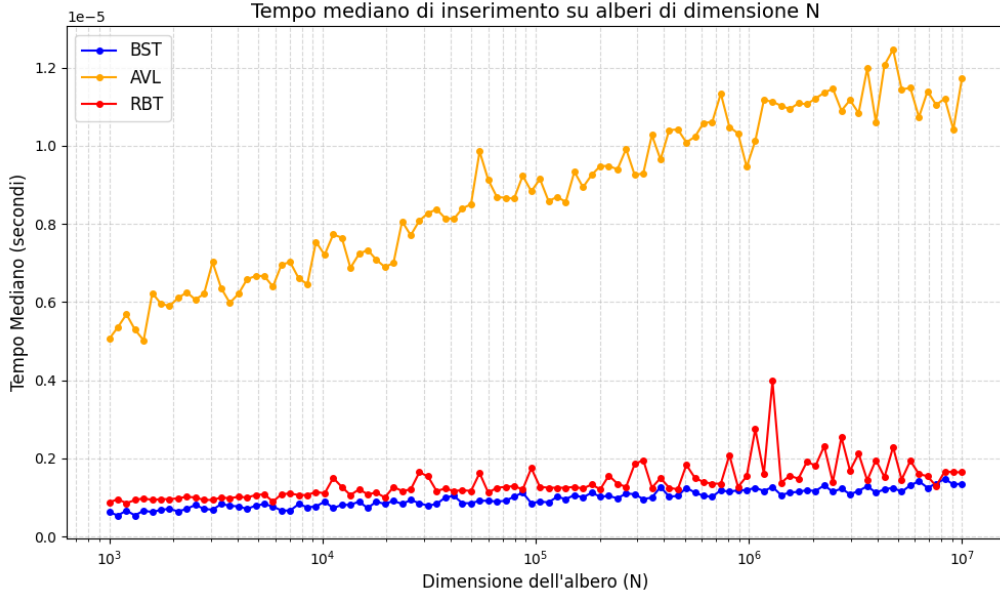
- **Dimensione Controllata:** Poiché l'eliminazione compensa esattamente l'inserimento precedente, viene garantito che prima di far partire il cronometro al giro successivo l'albero ritorni ad avere esattamente n chiavi.
- **Varietà Morfologica:** La continua espulsione e reinserimento di chiavi casuali altera costantemente e dinamicamente la topologia dei rami, modificando la struttura. In questo modo, le misurazioni avvengono su una famiglia di alberi le cui forme sono uniformemente distribuite nello spazio degli alberi possibili di dimensione n , evitando casi degeneri.

5 Risultati e Analisi

In seguito presentate e discusse le misurazioni eseguite in locale su MacBook Air M3 (con 16Gb di RAM), poi raccolti all'interno di un file in formato CSV ed elaborati all'interno di grafici.

5.1 Grafico Principale

Il grafico mostra il comportamento delle strutture dati nel caso medio, dove le chiavi vengono inserite con distribuzione casuale uniforme grazie al **KeyManager**.



L'asse delle ascisse esprime la dimensione N dell'albero sfruttando una scala logaritmica (da 1.000 a 10.000.000 elementi). Tramite la scala logaritmica si è in grado di mostrare una curva di complessità teorica pari a $\mathcal{O}(\log n)$, che assume l'aspetto grafico di una retta.

Dall'osservazione della figura, si nota:

- **Stabilità di BST e RBT:** Le progressioni relative a BST e RBT risultano costantemente schiacciate verso il basso lungo l'intero esperimento, denotando un'elevatissima efficienza d'esecuzione.
- **Inefficienza dell'AVL:** Di contro, la progressione associata all'albero AVL si distacca in modo netto spingendo visibilmente verso l'alto all'aumentare di N . Il vincolo di bilanciamento imposto dall'AVL (differenza di altezza massima pari a 1) si rivela estremamente inefficiente quando l'albero si fa altamente complesso. Ogni singola operazione innesca una catena di controlli e rotazioni correttive che, nel bilancio complessivo, pesano sul tempo complessivo.

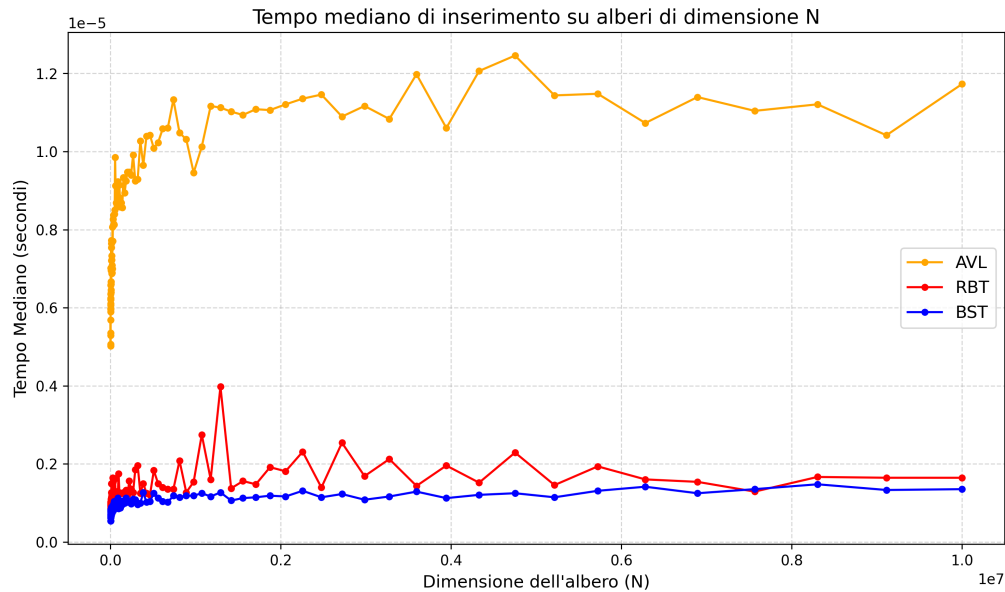
5.2 Grafico dell'analisi asintotica

Il grafico mostra, come il precedente, il comportamento delle tre strutture dati in seguito all'inserimento casuale di chiavi.

L'asse delle ordinate mantiene la misurazione in secondi su scala logaritmica, mentre l'asse delle ascisse cambia solamente la scala passando all'ordine 10⁷. Il grafico mostra una curva che, dopo un'accelerazione iniziale, flette e si stabilizza su una linea retta orizzontale.

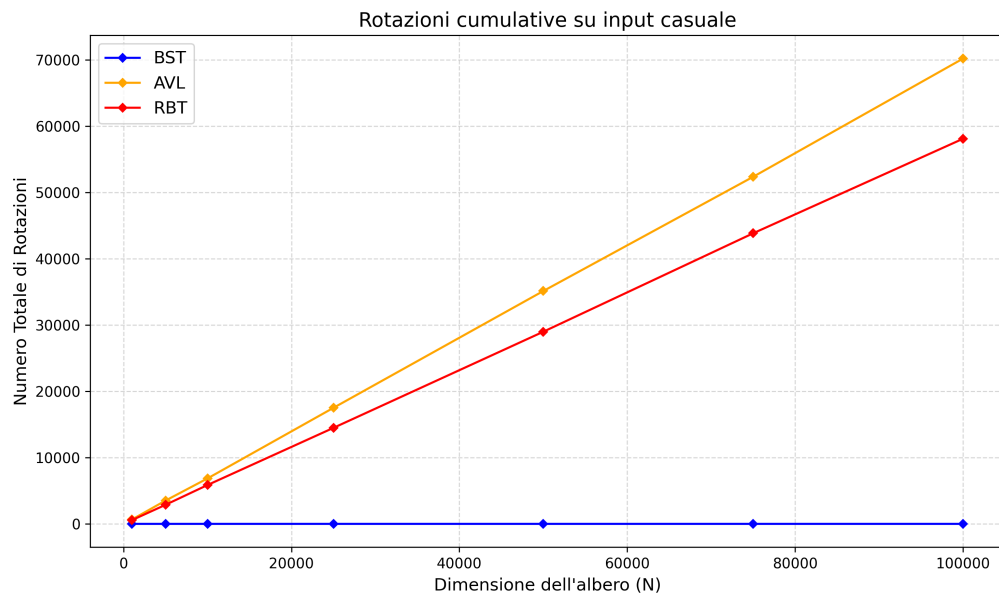
Dall'osservazione della figura, si nota:

- **Convergenza di BST e RBT:** Le curve di BST e RBT crescono inizialmente poco per poi "spianare", stabilizzandosi in modo netto su un andamento orizzontale sui grandi valori di N .
- **Divergenza dell'AVL:** Si nota invece che la progressione associata all'albero AVL, inizialmente sale molto più velocemente rispetto alle altre due strutture dati, per poi stabilizzarsi attorno a $1,1 \cdot 10^{-5}$ secondi. Questo dimostra graficamente che i vincoli dell'AVL rendono più difficile l'efficientamento.



5.3 Grafico Rotazioni Cumulative

Per la stampa di questo grafico l'inserimento è stato limitato alle chiavi [100, 500, 1000, 2500, 5000, 7500, 10000]. Viene mostrato il costo strutturale rappresentato come numero totale cumulativo di rotazioni eseguite dalle tre strutture dati in funzione di N durante il popolamento casuale uniforme. Si nota che

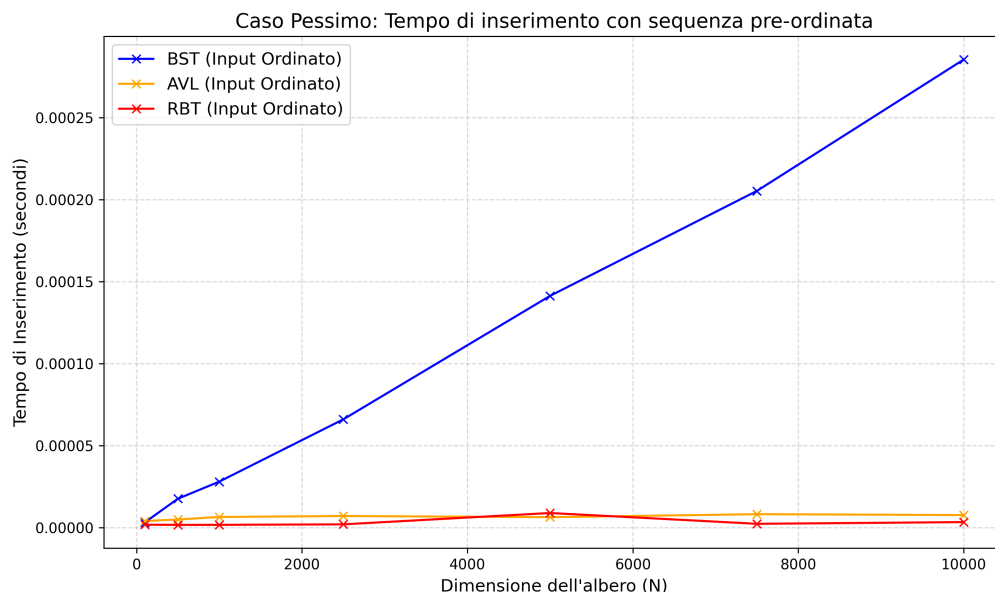


Dall'osservazione della figura, si nota:

- **Assenza di rotazioni nel BST:** La progressione del BST rimane costantemente ancorata allo zero per tutta la durata del test. L'algoritmo non esegue alcuna rotazione, confermando l'assenza totale di manutenzione nel caso medio.
- **Costo di bilanciamento dell'AVL rispetto al RBT:** L'albero AVL mostra una pendenza superiore rispetto alla variante Red-Black. A causa del vincolo sulle altezze, ogni operazione di inserimento ha un'alta probabilità di innescare riequilibri. Al contrario, l'albero Red-Black gestisce gran parte delle violazioni strutturali tramite la ricolorazione logica dei nodi (operazione molto più leggera per la CPU), ricorrendo alle rotazioni dei nodi con una frequenza inferiore.

5.4 Grafico Caso Pessimo

Per la stampa di questo grafico, come per il precedente, l'inserimento è stato limitato alle chiavi [100, 500, 1000, 2500, 5000, 7500, 10000]. Vengono mostrati i tempi di inserimento delle suddette chiavi pre-ordinate all'interno delle tre strutture dati.



Dall'osservazione della figura, si nota:

- **Collasso prestazionale del BST:** La progressione del BST devia con una pendenza marcatamente lineare verso l'alto già a ridosso di poche migliaia di nodi. Privo di logiche di riequilibrio, l'albero subisce passivamente l'ordinamento dell'input inserendo ogni chiave come figlio destro della precedente. La struttura degenera a tutti gli effetti in una lista concatenata, portando l'altezza a $h = \mathcal{O}(n)$, confermando il limite della struttura dati non bilanciata.
- **Resilienza strutturale di AVL e RBT:** Al contrario, le progressioni di AVL e RBT rimangono schiacciate a ridosso dello zero. Entrambe le strutture intercettano lo sbilanciamento attivando le rotazioni dei nodi, preservando l'efficienza asintotica logaritmica $\mathcal{O}(\log n)$ anche a fronte del peggior input possibile. Il confronto mostra come il Red-Black si mantenga leggermente più efficiente dell'AVL grazie alla minore complessità delle operazioni di ricolorazione rispetto alle rotazioni rigide.

6 Conclusioni

Lo studio ha permesso di analizzare il comportamento delle tre principali varianti di alberi binari di ricerca (BST, AVL e RBT), confermando le proprietà teoriche attraverso misurazioni su larga scala.

Le misurazioni effettuate evidenziano come la scelta della struttura dati ottimale sia strettamente dipendente dalla natura del carico di lavoro e dalla distribuzione dei dati in ingresso:

- Il **BST semplice** si è rivelato il più efficiente in scenari con input casuale uniforme. L'assenza di esecuzioni di riequilibrio annulla l'overhead sui puntatori, sfruttando la proprietà spontanea di bilanciamento indotta dalla casualità dei dati. Tuttavia, dal caso pessimo ne è stata evidenziata la fragilità, mostrando un calo di performance portando ad una complessità di $\mathcal{O}(n)$ non appena l'input presenta ordinamento crescente.
- L'**Albero AVL**, pur garantendo la complessità logaritmica nello scenario peggiore, risente fortemente del vincolo sulle altezze. Nelle applicazioni ad alta frequenza di scrittura, l'elevato numero di rotazioni necessarie a mantenere la struttura bilanciata introduce un rallentamento della CPU che ne penalizza i tempi rispetto alle altre strutture dati.

- L'**Albero Red-Black** si è dimostrato il miglior compromesso complessivo. Riuscendo a controllare lo sbilanciamento meglio rispetto all'AVL, riducendo il ricorso alle rotazioni dei nodi, resolvendo gran parte delle violazioni tramite ricolorazioni rosso-nere. Questa flessibilità consente di mantenere prestazioni ottime sia nel caso medio che di fronte a input degeneri, confermandolo come la struttura più robusta e versatile per scopi generali.

In conclusione, l'analisi ha confermato che i meccanismi di bilanciamento automatico rappresentano un vero e proprio "salvagente" dal punto di vista della complessità: pur introducendo un costo di manutenzione (evidente nell'AVL), essi salvano dalle degenerazioni a $\mathcal{O}(n)$ nel caso pessimo, garantendo una sicurezza permettendo l'utilizzo nei sistemi software moderni (RBT nel File System di Linux) su scale di dati critiche.